CIT CAMPUS SCHEDULING PLATFORM

Milestone 2F

Database Runtime Preparation & Source Contract Validation

Engineering checkpoint · Package v0.4 · 31 October 2026

Date: 31 October 2026

Status: PARTIALLY COMPLETE — local runtime bootstrap and source-contract/ORM foundations implemented; live PostgreSQL execution and transactional import remain blocked/pending.

1. Purpose and continuity

This milestone continues the backend-first sequence established in the PRD, source audit, ERD, API/domain contract, consistency review, revised database blueprint v0.2 and DDL baseline v0.3. It does not restart architecture decisions.

Preserved boundaries:

- Backend and database first. The frontend is not part of this milestone.
- The system must have real backend workflows; placeholder endpoints and dummy data are not accepted as completion.
- Campus Navigator is deferred to the teammate's separate module. This package adds no map UI, map processing, indoor/outdoor routing, geometry pipeline or navigation tables.
- The official starter JSON and `validator.py` remain authoritative over assumptions in this application.
- The four competition roles remain `admin`, `department_coordinator`, `faculty`, and `club`; no fifth role is introduced.

2. Work completed in v0.4

2.1 Local PostgreSQL runtime bootstrap

- Added `docker-compose.yml` for a disposable PostgreSQL 16 development service.
- Added health check, named data volume, local database/user and configurable password.
- Added `.env.example`; credentials are development-only placeholders and must not be committed as real secrets.
- Added a reproducible runbook.

2.2 Strict source contract

- Added Pydantic 2 source models for all canonical JSON root sections and their nested records.
- Root keys are checked exactly. Unexpected root/nested keys fail validation rather than being silently ignored.
- Cross-reference checks cover session→course/faculty/groups, unavailability→room/day/period, booking→day/group, cancellation→booking, and exam→course/group.
- Added deterministic source SHA-256 calculation and observed-count/ID extraction helpers.
- A real fixture discrepancy was found and fixed: faculty preference entries include an optional `note` field. The schema now preserves it.

2.3 SQLAlchemy 2.x mapping foundation

- Added typed declarative mappings for 15 tables in the source-import foundation: institutes, departments, dataset versions/policy, calendar versions/days/periods, rooms/features/assignments/allowed uses, faculty, student groups, courses and teaching sessions.

- The mapping is intentionally partial. It is not a claim that all 78 database tables have ORM classes.
- PostgreSQL dialect DDL compilation of the current 15 mapped tables passes.
- Alembic remains the authoritative schema migration mechanism; `metadata.create_all()` is not a deployment strategy.

2.4 Migration execution hardening

- Updated the initial migration to execute ordinary DDL statements individually, avoiding reliance on a single multi-command `execute()` call with psycopg 3.
- This is a code-level mitigation based on driver execution semantics; it is not live database proof.
- Alembic offline SQL rendering succeeds and emits 78 application tables plus Alembic's own version table.

3. Fixture verification

Source: `CrunchWeek_StarterPack/data/cit_public.json`.

Section	Observed records
Rooms	27
Room unavailability blocks	2
Faculty	40
Student groups	14
Courses	62
Teaching sessions	189
Booking requests	25
Cancellations	1
Exams	41

These are observed fixture values, not constants to hardcode in production logic.

The source checksum is computed at runtime by `load_source`; it is not used as a substitute for per-record identity and relationship validation.

4. Verification results

Check	Result	What it proves
Official JSON source contract tests	PASS	Current supplied fixture parses against the strict source models and cross-reference checks
Fixture record count/ID uniqueness tests	PASS	Expected observed fixture counts and unique IDs are preserved
SQLAlchemy PostgreSQL dialect compilation	PASS	Current 15 ORM table mappings compile into PostgreSQL DDL
Alembic offline upgrade rendering	PASS	The migration script can render SQL for the 78-table baseline
Live PostgreSQL migration	NOT RUN	No Docker, PostgreSQL server/client binaries or local server were available in this environment
SQL acceptance suite against PostgreSQL	NOT RUN	Requires a live PostgreSQL instance
ORM-to-DDL parity	NOT COMPLETE	Only the initial 15-table ORM foundation is mapped; 63 tables remain unmapped
Transactional source import	NOT IMPLEMENTED	Must follow live migration and full relationship mapping

Check	Result	What it proves
Re-import/rollback/activation tests	NOT RUN	Requires import service and live PostgreSQL

5. Critical implementation observations

- **Offline rendering is not database validation.** No claim is made that the 78-table DDL has executed successfully against PostgreSQL.
- **ORM coverage is partial.** The 15 mappings are an import foundation, not complete model coverage.
- **The initial schema has cross-dataset integrity gaps.** Several child relationship tables use single-column FKs. A hardening migration must add dataset-scoped composite constraints where the parent and child carry dataset identity.
- **Source departments are text codes.** The source does not provide a canonical department master list. Import mapping must create/resolve department records deterministically without inventing a registry.
- **Preference semantics are source-sensitive.** The fixture provides `type`, `days`, `periods`, and optional `note`; preserve the note even though the validator's soft score consumes only avoid-type periods.
- **Calendar period IDs are source numbers.** Do not assume periods are contiguous or infer lunch from missing periods; persist explicit lunch and calendar policy.
- **Database extension permissions:** the baseline requests `pgcrypto` for `gen_random_uuid()`. On managed PostgreSQL, confirm extension availability/permissions before migration.
- **No authentication provider has been frozen.** Local password vs external OIDC remains an explicit decision; do not store plaintext passwords or implement a weak custom auth flow by default.

6. Rules verified directly against official validator.py

Timetable hard constraints

- Every source session must appear exactly once; unknown or duplicate assignments are violations.
- Teaching sessions must fit P1–P9 and must not cross the lunch boundary.
- Labs must start at the configured lab start periods (fixture: P2 or P6).
- Fixed slots must be preserved.
- Room capacity and allowed room type must fit.
- Accessibility requirement must be satisfied.
- Room and faculty unavailability are hard violations.
- Room, faculty and mandatory student-group overlaps are hard violations.
- Basket sessions must share a common day/start time; the validator's group overlap key treats sessions in the same basket as one shared conflict key.
- Faculty daily teaching hours must not exceed the faculty record's `max_hours_per_day` (default in validator context: 4).

Soft score formulas

- S1: 2 × each avoid-preference period hit by an assignment.
- S2: 1 × each missing period between the earliest and latest occupied period for a group/day.
- S3: 3 × each group-hour above 6 teaching hours per day.
- S4: 3 × each additional lecture of the same course for a group/day; basket-course repeats also contribute.
- S5: 1 per assigned session whose attendance/size is below 40% of room capacity.
- S6: 1 per assigned session that occupies P9.
- Exam soft penalty: 5 × each additional exam on the same day for a student group.

Booking priority and validation

- Priority order is read from the dataset's `priority_order`; do not duplicate a hardcoded ranking in business logic.

- Effective category is constrained by `role_category_cap`; the validator's effective-category function promotes a request to the lower-priority (larger index) of requested category and role cap.
- Within equal effective category, earlier `submitted_at` outranks later.
- Official category score weights: institute event 5, guest lecture 4, department event 3, club event 2.
- Cancelled requests cannot be approved or offered an alternative.
- Approved requests must fit requested windows; an out-of-window slot must be represented as an alternative offer.
- Request duration, room event eligibility, type, capacity, features/accessibility, room unavailability, timetable room occupancy and mandatory group occupancy are checked.
- A rejected higher-priority request that could have used a slot occupied by a lower-priority approved request is reported as a priority inversion.

Exams

- Every exam must be scheduled exactly once.
- Exam day and session must be valid.
- Rooms must allow exam use; effective seat count is `sum(room.capacity // 2)`.
- Room and group exam overlaps are hard violations.
- A group with multiple exams on one day incurs the exam soft penalty above.

7. Decisions still open (not silently resolved)

- Authentication provider and account provisioning.
- Booking reviewer routing by category, role and scope.
- Whether alternative-offer acceptance/decline appears in the official export contract.
- Exact semantics for recurring `series_id` requests.
- Whether a booking can span multiple rooms (not supported by the current export structure; defer unless organiser confirms).
- Whether timetable generation/publishing requires Admin approval (recommended policy, organiser confirmation needed).
- Whether Department Coordinators can directly edit or only submit scoped proposals (proposal-only is the safe default pending confirmation).
- Student read-only access and identity verification policy.
- Exact organiser interpretation of dataset versioning and export serialization.

8. Next gate — must happen before declaring database foundation ready

- Start PostgreSQL 16 using the included Compose file on a machine with Docker.
- Apply the initial Alembic migration against a fresh disposable database.
- Run `tests/schema_acceptance.sql` and inspect actual catalog objects.
- Resolve any PostgreSQL runtime errors; do not edit an already-applied migration—create a forward corrective migration.
- Complete ORM mappings for every table used by import, scheduling, booking, exams, approvals and change management.
- Add dataset-scoped composite FKs and validate all cross-dataset references.
- Implement source import as one transaction with ID maps, strict relationship checks, counts/ID-set reconciliation and rollback tests.
- Add activation as a separate locked transaction and test one-active-dataset invariant.
- Compare generated exports byte/structure-wise against the organiser's examples and run the official validator.

Only after these gates should the platform move to functional timetable generation and the remaining backend workflows.